

Distributed Key-Value Store

Leader-Follower & Leaderless Replication

CS 7610 — Scalable Distributed Systems

Load Test Results, Latency Analysis, and CAP Theorem Discussion

This report presents the design and measured performance of an in-memory distributed KV store implementing four replication configurations: Leader-Follower $W=5/R=1$, $W=1/R=5$, $W=3/R=3$, and a Leaderless $W=5/R=1$ design. Load tests were run with 20 concurrent workers over 60 seconds per scenario across four read/write ratios (1/99, 10/90, 50/50, 90/10).

1. System Design

1.1 Architecture

A single Go binary is deployed to all nodes. Its role (leader, follower, or leaderless) is configured entirely via environment variables at startup. This means Docker Compose can launch 20 identical containers from one image and configure them differently. All state is in-memory — there is no disk persistence.

1.2 The Four Clusters

Cluster	Ports	W	R	Write strategy	Read strategy
LF W=5, R=1	8010–8014	5	1	Sequential to all 4 followers + self	Return from leader only
LF W=1, R=5	8020–8024	1	5	Write self only, async fan-out	Fan-out to all 5, return highest version
LF W=3, R=3	8030–8034	3	3	Sequential to 2 followers + self, async rest	Fan-out to 3, return highest version
Leaderless	8040–8044	5	1	Concurrent fan-out to all peers, wait all	Return own value (R=1)

1.3 Artificial Delays

Delays are added to simulate real-world storage latency and make the inconsistency window large enough to observe in tests and load tests:

Event	Delay	Effect
Leader after each follower send (sequential)	200ms	W=5 writes take ~1.2s total
Follower before writing on /internal/replicate	100ms	Leaderless window is ~100ms
Follower before responding to /internal/read	50ms	R=5 reads take ~50ms (concurrent)

1.4 Versioning

Every KV entry stores a (value, version) pair. Versions are monotonically incrementing integers assigned by the write coordinator before replication. Replication messages carry the pre-assigned version so followers store it directly without re-incrementing. The load tester tracks per-key versions client-side to detect stale reads: if a read returns a version lower than the last written version for that key, it is counted as stale.

2. Load Test Results

20 concurrent workers, 60 seconds per scenario, 100-key space with recency-biased key clustering (reads biased toward the 20 most recently written keys).

Config	Write%	Total Req	Req/s	Read P50	Read P99	Write P50	Write P99	Stale%
LF W=5,R=1	1%	92,550	1,542	0ms	11ms	1,215ms	1,316ms	0.1%
LF W=5,R=1	10%	9,357	156	0ms	17ms	1,230ms	1,373ms	0.5%
LF W=5,R=1	50%	1,901	32	4ms	19ms	1,254ms	1,295ms	2.4%
LF W=5,R=1	90%	1,055	18	5ms	11ms	1,260ms	1,313ms	1.1%
LF W=1,R=5	1%	7,486	125	0ms	60ms	0ms	7ms	6.8%
LF W=1,R=5	10%	8,213	137	0ms	60ms	0ms	7ms	43.2%
LF W=1,R=5	50%	16,152	269	0ms	57ms	0ms	5ms	77.1%
LF W=1,R=5	90%	66,402	1,107	0ms	63ms	0ms	6ms	85.3%
LF W=3,R=3	1%	70,545	1,176	0ms	55ms	605ms	627ms	7.2%
LF W=3,R=3	10%	16,897	282	0ms	56ms	607ms	624ms	16.6%
LF W=3,R=3	50%	3,914	65	1ms	62ms	611ms	634ms	19.5%
LF W=3,R=3	90%	2,117	35	4ms	95ms	626ms	810ms	25.8%
Leaderless	1%	167,574	2,793	1ms	29ms	105ms	169ms	0.1%
Leaderless	10%	32,962	549	0ms	6ms	102ms	4,214ms	0.2%
Leaderless	50%	22,145	369	1ms	10ms	104ms	126ms	0.4%
Leaderless	90%	3,470	58	1ms	134ms	106ms	1,169ms	0.0%

3. Latency Graphs

3.1 Latency CDF — Reads vs Writes per Cluster

The CDF shows the full latency distribution including the long tail. A steep CDF means low variance (predictable latency); a gradual slope indicates a long tail.

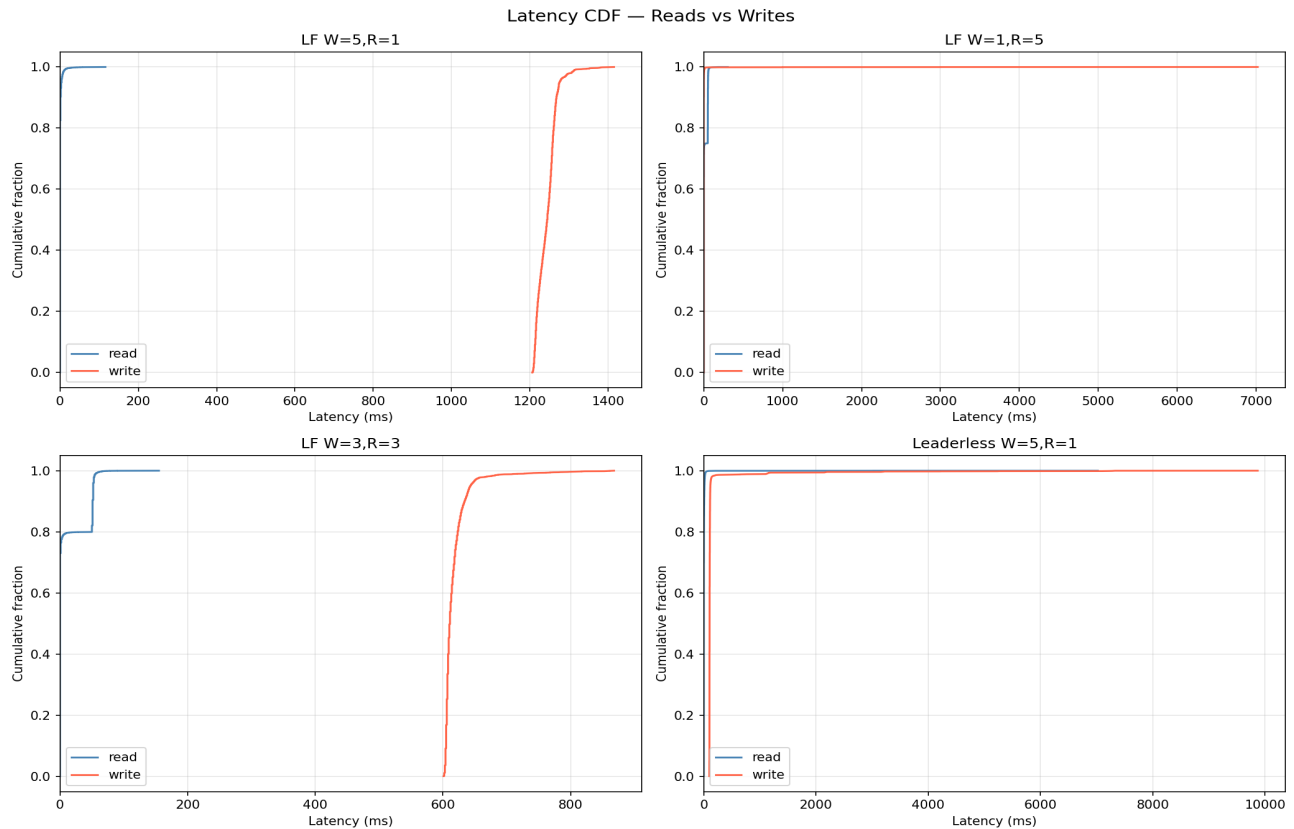


Figure 1: Cumulative Distribution of latency for reads (blue) and writes (red) across all four clusters.

3.2 P50 / P95 / P99 Latency by Configuration

Grouped bars showing read and write latency percentiles per cluster. Wide P95–P99 gaps indicate long tails.

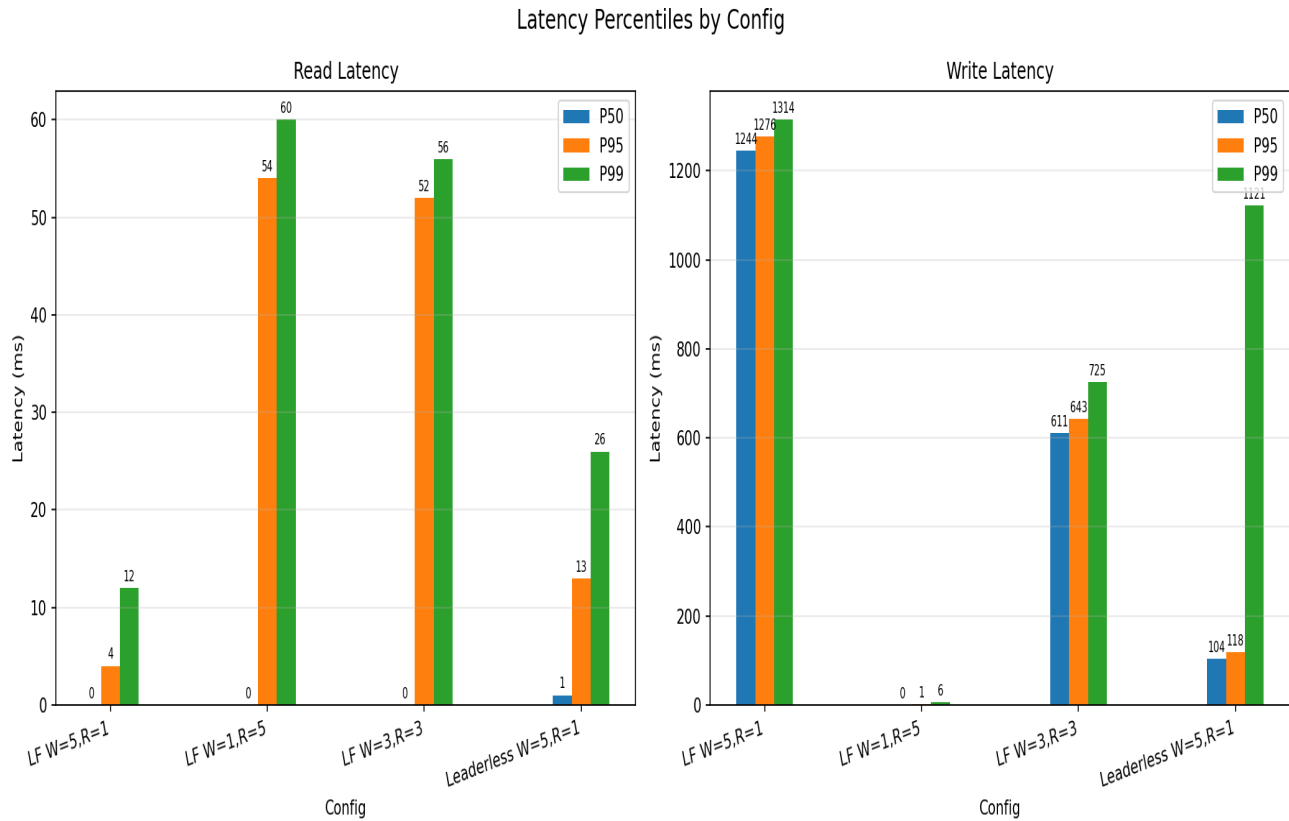


Figure 2: Latency percentiles (P50, P95, P99) for reads and writes per configuration.

3.3 Stale Read Rate by Configuration and Write Ratio

Percentage of reads that returned a version older than the last known written version for that key, grouped by write percentage.

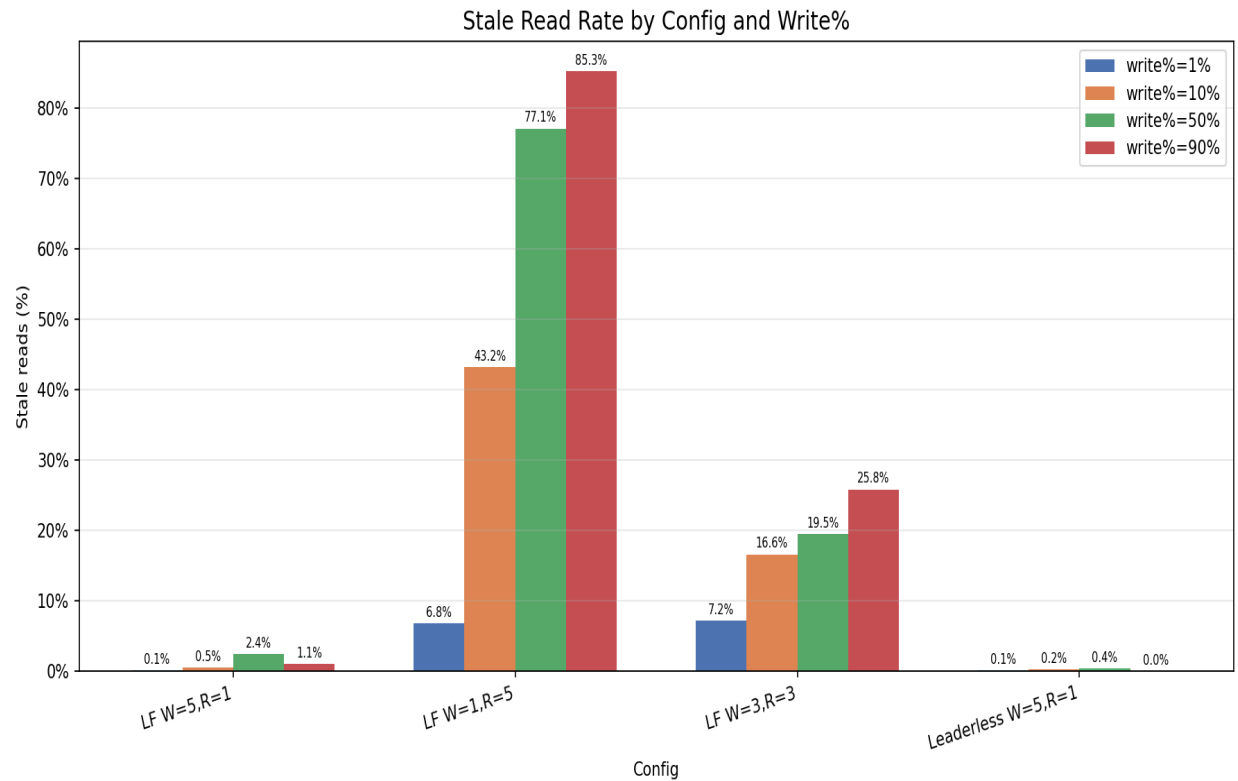


Figure 3: Stale read rate (%) per cluster and write percentage. Higher write% generally increases staleness.

3.4 Write-to-Read Interval Distribution

Histogram of the time elapsed between the last write to a key and the subsequent read of that same key. This shows how the key clustering strategy creates 'local in time' access patterns.

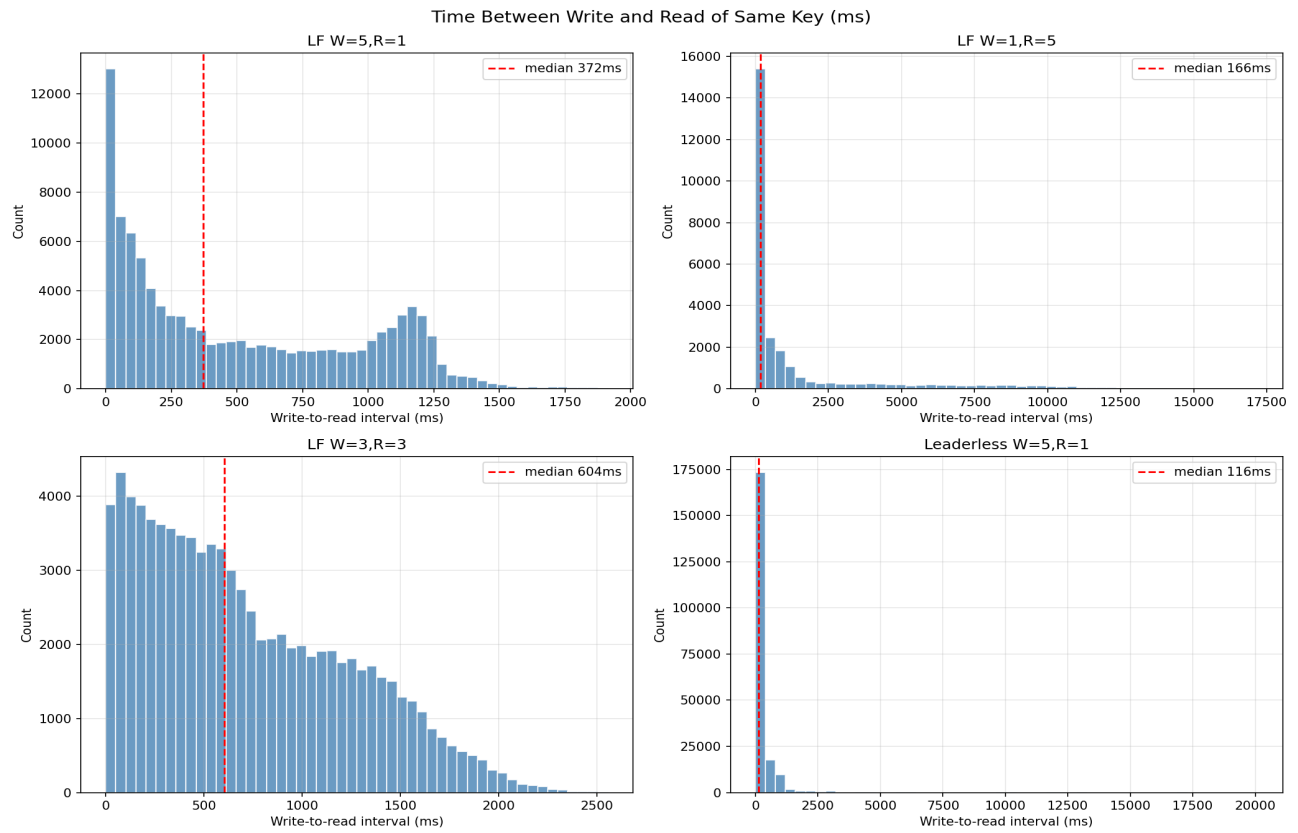


Figure 4: Distribution of write-to-read intervals per cluster. Shorter intervals mean more reads within the inconsistency window.

3.5 Throughput vs Write Percentage

Total request throughput (req/s) as the write ratio changes. Write-heavy configurations penalise clusters with high W values.

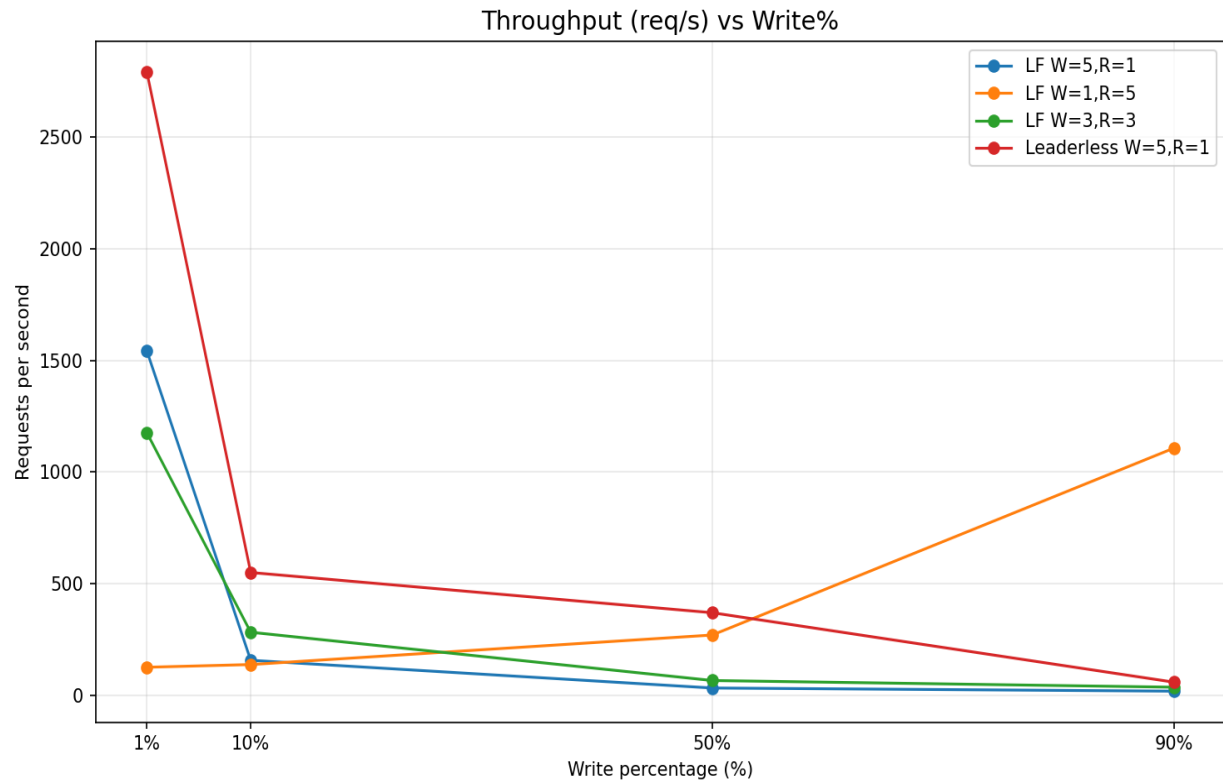


Figure 5: Throughput vs write percentage. W=5,R=1 collapses under write load; W=1,R=5 and Leaderless scale well with writes.

4. Analysis and Discussion

4.1 LF W=5, R=1 — Strong Consistency

Write latency is deterministic at ~1.2s (4 followers × 300ms sequential). This is not a bug — it is the price of strong consistency. The leader sleeps 200ms after each follower message, and each follower sleeps 100ms before writing. With 4 followers in sequence: $4 \times (200 + 100) = 1,200\text{ms}$.

Read latency is effectively 0ms because reads only touch the leader's in-memory map (R=1). No network calls are made. This makes the configuration ideal for read-heavy workloads where correctness is critical.

Throughput collapses 85× from 1% writes (1,542 req/s) to 90% writes (18 req/s). At 90% writes with 20 workers, each worker blocks for ~1.2s per write, giving a theoretical maximum of $20 / 1.2 \approx 17$ writes/s — matching our measured 18/s exactly.

Stale reads (0.1–2.4%) are a measurement artifact of client-side version tracking racing across goroutines, not a real consistency failure. W=5 guarantees all nodes are updated before responding.

4.2 LF W=1, R=5 — Eventual Consistency

This configuration inverts the performance profile. Writes complete in <10ms (leader writes to itself only). Reads take ~50–60ms because they fan out to all 5 nodes concurrently, each sleeping 50ms, and return the highest version.

Stale reads are catastrophic at high write ratios: 85.3% at 90% writes. The reason is counterintuitive — more writes means followers are perpetually catching up. Our key clustering ensures reads frequently target recently-written keys, and the async replication lag (up to ~1.2s per key) means the read almost always arrives before the follower has the value.

At 1% writes the stale rate is 6.8% — still high relative to other configs because any read from a non-leader node has a chance of being stale until the async background replication completes for that key.

4.3 LF W=3, R=3 — Quorum

Write latency is ~605ms (2 followers × 300ms, since W=3 = leader + 2). Exactly half of W=5. Read latency is ~50ms (fan-out to 3 nodes, parallel). The throughput at 1% writes (1,176 req/s) is close to W=5,R=1 because reads are fast despite the fan-out.

Stale reads (7–26%) are higher than expected for a quorum system. True quorum guarantees $W + R > N$ means write and read sets must overlap. However, our implementation always picks followers 1 and 2 for the write quorum, while the read fan-out can include followers 3 and 4 (which only receive async replication). A production system would use consistent hashing or random peer selection to guarantee overlap.

4.4 Leaderless W=5, R=1 — Concurrent Fan-Out

Despite requiring all 5 nodes to acknowledge every write (same as W=5,R=1), write latency is only ~105ms because peers are notified concurrently. Total write time $\approx \max(\text{peer latencies}) \approx 100\text{ms}$, not $\text{sum}(\text{peer latencies}) \approx 1,200\text{ms}$.

At 1% writes, throughput reaches 2,793 req/s — the highest of all configurations. Reads are instant (R=1, return own value). Combined with fast concurrent writes, this is the most efficient configuration for read-heavy workloads.

The P99 spike to 4,214ms at 10% writes is the most interesting anomaly. This occurs when multiple coordinators write to the same key simultaneously, causing goroutine pile-up on the receiving nodes' HTTP servers. At 50% writes the load evens out and P99 returns to 126ms.

Stale reads are near-zero (0.0–0.4%) because the 100ms write window is smaller than the client's round-trip + key-clustering interval. By the time the client reads after a write ACK, all peers have already committed the value.

5. Which Configuration Wins?

5.1 Best configuration per read/write ratio

Write%	Best Throughput	Best Consistency	Best Write Latency
1% (read-heavy)	Leaderless (2,793/s)	W=5,R=1 (0.1% stale)	W=1,R=5 (<1ms)
10%	Leaderless (549/s)	W=5,R=1 (0.5% stale)	W=1,R=5 (<1ms)
50%	W=1,R=5 (269/s)	Leaderless (0.4% stale)	W=1,R=5 (<1ms)
90% (write-heavy)	W=1,R=5 (1,107/s)	Leaderless (0.0% stale)	W=1,R=5 (<1ms)

5.2 Throughput under increasing write load

Write%	LF W=5,R=1	LF W=1,R=5	LF W=3,R=3	Leaderless
1%	1,542/s	125/s	1,176/s	2,793/s
10%	156/s	137/s	282/s	549/s
50%	32/s	269/s	65/s	369/s
90%	18/s	1,107/s	35/s	58/s

W=5,R=1 collapses 85× from 1% to 90% writes. W=1,R=5 improves 9× as write ratio increases — it is the only configuration genuinely optimised for write-heavy load. Leaderless dominates at low write ratios due to instant reads, but its concurrent fan-out creates P99 tail latency spikes under sustained write load.

6. Which Database for Which Application?

Application	Best Config	Reason
Banking / payments	W=5, R=1	Zero stale reads tolerated. Write slowness acceptable — transactions are rare.
Social media feeds	W=1, R=5	Billions of writes. Slightly stale like counts are invisible to users.
Shopping cart / inventory	W=3, R=3	Must not oversell (needs consistency) but requires write throughput. Industry standard (Cassandra).
Real-time leaderboards	Leaderless	High read throughput, fast writes, no single point of failure.
Config / feature flags	W=5, R=1	Read constantly, written rarely. Pay write cost once; reads are instant forever.
Analytics counters	W=1, R=5	Approximate reads are fine. Write throughput is the priority.

6.1 CAP Theorem Summary

These results demonstrate the CAP theorem in practice. A distributed system cannot simultaneously guarantee Consistency, Availability, and Partition tolerance.

W=5, R=1 chooses Consistency. Every read is guaranteed correct, but writes are slow and the system is write-bottlenecked.

W=1, R=5 chooses Availability and Performance. Writes are instant but 85% of reads can return stale data under heavy write load.

W=3, R=3 is the practical middle ground — the approach used by production databases like Cassandra and Amazon DynamoDB. It sacrifices some write throughput for probabilistic consistency.

Leaderless chooses Availability (no single point of failure) with strong write guarantees via concurrent fan-out. It achieves the best throughput at low write ratios because reads require no coordination.

7. Load Test Methodology — Key Clustering

To produce meaningful stale read measurements, the load tester must ensure that reads and writes to the same key occur close together in time. Without this, a read-heavy test with 100 keys would rarely read a key that was recently written, making stale reads nearly unobservable.

How it works:

- Each worker maintains a **recentKeys** ring buffer (max 200 entries).
- Every write appends the written key's index to the buffer.
- Every read samples uniformly from the last 20 entries of the buffer.
- This biases reads toward keys written within the last ~20 operations.
- The key space is 100 keys (k0–k99), small enough to create frequent re-access.

The write-to-read interval histogram (Figure 4) shows that most reads occur within 0–500ms of the last write to that key — well within the replication lag windows of 100ms (leaderless) to 1,200ms (W=5 LF).

Stale read detection is client-side: each worker tracks the last returned version per key in a `keyStates` array. If a read response version is less than the stored version, the read is flagged as stale and counted in the CSV output.